

Contents

4. Arquitecturas híbridas LLM+GOAP y modelos de lenguaje compactos	1
4.1 GOAP como mecanismo de ejecución táctica y sus límites semánticos	1
4.2 La arquitectura de dos niveles: LLM para deliberar, planificador para ejecutar	1
4.3 La latencia como motivación central y estrategias de balance	2
4.4 Modelos de lenguaje compactos como vía de viabilidad	2
Referencias	3

4. Arquitecturas híbridas LLM+GOAP y modelos de lenguaje compactos

La literatura reciente converge en un patrón de diseño de dos niveles para dotar a los NPC de comportamiento creíble sin sacrificar el rendimiento en tiempo real: un componente de Modelo de Lenguaje (LLM) se encarga de la deliberación semántica de alto nivel (interpretar la situación y fijar intenciones/objetivos) mientras que un planificador clásico —típicamente GOAP o un planificador automático equivalente— traduce esas intenciones en secuencias de acciones ejecutables y físicamente viables dentro del motor [1]. Este apartado sintetiza el funcionamiento de GOAP, la arquitectura híbrida propuesta, la motivación por la latencia y el rol de los modelos compactos.

4.1 GOAP como mecanismo de ejecución táctica y sus límites semánticos

Goal-Oriented Action Planning (GOAP) es una técnica de planificación en tiempo real popularizada por Jeff Orkin en el videojuego *F.E.A.R.* (2005), que adapta el sistema STRIPS (1971) al contexto de los juegos [7]. En GOAP el diseñador define un conjunto de acciones, cada una con precondiciones y efectos, junto con metas (goals) y un estado del mundo (world state) representado como pares clave-valor. El planificador construye dinámicamente, en tiempo de ejecución, la secuencia de acciones de menor costo que lleva del estado inicial al estado que satisface la meta; para ello trata cada estado del mundo como un nodo y emplea el algoritmo A* para hallar la “ruta” óptima entre estados [7]. Esto desacopla las acciones entre sí y permite reemplazar cadenas rígidas de comportamiento (máquinas de estados finitos o árboles de comportamiento) por planes emergentes y adaptativos, con la posibilidad de re-planificar cuando el estado cambia.

La limitación central de GOAP es que su razonamiento es puramente sintáctico-simbólico: el planificador solo manipula símbolos con precondiciones y efectos, pero carece de comprensión semántica de lo que esos símbolos significan. Un desarrollador lo ilustra al señalar que una acción genérica como “PickupItem” es “semánticamente vacía”: el planificador no sabe si el ítem es comida, un arma o material de fabricación, y por ello no puede construir cadenas de dependencia significativas [8]. En términos más generales, los sistemas simbólicos dependen exclusivamente de reglas predefinidas, carecen de capacidad de generalización de sentido común y no pueden inferir más allá de su especificación original, resultando frágiles ante escenarios no anticipados [9]. Esta rigidez es precisamente la que motiva incorporar un LLM como capa deliberativa.

4.2 La arquitectura de dos niveles: LLM para deliberar, planificador para ejecutar

Puerta-Merino y Sabater-Mir [1] proponen una arquitectura que integra un LLM para la toma de decisiones con un planificador automático clásico capaz de generar planes correctos (sound plans) para esa decisión, con el objetivo de que el agente decida en situaciones no anticipadas durante el diseño. La arquitectura se organiza en tres módulos: (i) un **Reasoner** que consulta al LLM para derivar objetivos a partir de la percepción del entorno, apoyándose en una estructura de “memorias” que incluye rasgos de personalidad y percepciones relevantes; (ii) un **Planner** que toma el objetivo seleccionado y lo traduce a un problema de Planificación Automática (estado inicial, condiciones meta y acciones) expresado en PDDL, invocando un solver que devuelve la secuencia de acciones ejecutables, y que re-planifica cuando cambia el contexto o la meta; y (iii) un **Interface** que actúa como capa de comunicación con el entorno simulado, procesando actualizaciones de estado y ejecutando las acciones secuencialmente [1]. Aunque los autores emplean un planificador PDDL genérico (mediante la librería Unified-Planning) y no un GOAP nominal, el patrón es idéntico al que interesa a esta tesis: el LLM aporta la deliberación semántica de alto nivel (el “qué” y el “por qué”) y el planificador simbólico garantiza la ejecución táctica correcta y físicamente viable (el “cómo”). La validación se realizó sobre “The Firefighter Problem”, donde un agente bombero interactúa con un vehículo en llamas y una persona atrapada, mostrando comportamiento diverso según rol y personalidad [1]. Este mismo esquema jerárquico

—LLM como planificador de alto nivel y un controlador/planificador clásico de bajo nivel— es hoy dominante también en robótica, donde el LLM actúa como “traductor semántico” y la búsqueda combinatoria y la viabilidad física se delegan a planificadores clásicos [12].

4.3 La latencia como motivación central y estrategias de balance

El motivo por el que no se ejecuta un LLM completo para toda la lógica del NPC es el costo computacional y la latencia. Kang, Zhang, Cai, Xu, Krishna, Du y Weissman [2] presentan el primer estudio sistemático del compromiso latencia-calidad en tareas de decisión en tiempo real, e introducen dos benchmarks: HFTBench (trading de alta frecuencia) y StreetFighter (juego competitivo basado en la plataforma DIAMBRA). Su hallazgo clave es que el balance óptimo latencia-calidad varía según la tarea, y que sacrificar calidad por menor latencia puede mejorar significativamente el desempeño final [2]. Para explotarlo proponen **FPX**, un marco adaptativo de precisión mixta que selecciona dinámicamente el tamaño de modelo y el nivel de cuantización según la demanda en tiempo real, conmutando a nivel de operador entre kernels de multiplicación de matrices FP8 y FP4: aplica FP4 solo a las capas lineales tolerantes a compresión agresiva y conserva FP8 en las más sensibles [2]. Los resultados son concretos: FPX mejora la tasa de victorias “hasta 80% en Street Fighter y aumenta el rendimiento diario hasta 26.52% en trading” (“*improving win rate by up to 80% in Street Fighter and boosting daily yield by up to 26.52% in trading*”); la mejor configuración en Street Fighter es un modelo de 3B con 30% de capas en FP4 y 70% en FP8, y en trading un modelo de 14B con 20% de sus capas lineales comprimidas a FP4 y el resto en FP8 [2]. El estudio también matiza que “más rápido” no siempre es mejor: reducir la latencia de modelos débiles (7B) puede perjudicar el desempeño, y en Street Fighter cada acción toma un tiempo fijo (~5 acciones por segundo, 200 ms por acción), de modo que optimizar por debajo de ese umbral no aporta beneficio [2]. Esto confirma que en decisiones sensibles a la latencia conviene reservar el razonamiento pesado del LLM para la deliberación esporádica de alto nivel y delegar la ejecución frecuente a un componente rápido y determinista como GOAP. Otras estrategias reportadas en la literatura para balancear velocidad y precisión incluyen la compresión de modelos (cuantización, poda y destilación), el *caching* [10] y el *speculative decoding*, técnica en la que un modelo “borrador” pequeño propone tokens que el modelo grande verifica en paralelo: Leviathan, Kalman y Matias demostraron una aceleración de 2x–3x con salidas idénticas a las del modelo estándar [13], y Chen et al. reportaron un *speedup* de 2x–2.5x en Chinchilla 70B sin comprometer la calidad de las muestras [14].

4.4 Modelos de lenguaje compactos como vía de viabilidad

La viabilidad de esta integración dentro de un motor de juego descansa en los modelos de lenguaje compactos (small/compact LLMs) optimizados para inferencia bajo restricciones de latencia. Belcak y colaboradores (NVIDIA Research) [3] argumentan que los modelos pequeños (SLM) “son suficientemente capaces, inherentemente más adecuados y necesariamente más económicos para muchas de las invocaciones en sistemas agénticos” (“*sufficiently powerful, inherently more suitable, and necessarily more economical for many invocations in agentic systems*”), definiendo como SLM, en 2025, a la mayoría de los modelos por debajo de 10 000 millones (10B) de parámetros, y sosteniendo que habilitan despliegues en el borde y on-device con mejor privacidad y latencia [3]. Ejemplos representativos incluyen Phi-3-mini de Microsoft, un modelo de 3.8B de parámetros entrenado sobre 3.3 billones de tokens que alcanza 69% en MMLU y 8.38 en MT-bench, con desempeño comparable a modelos como Mixtral 8x7B y GPT-3.5 pese a ser lo bastante pequeño para desplegarse en un teléfono [4]; y TinyLlama, un modelo compacto de 1.1B de parámetros construido sobre la arquitectura de Llama 2 y optimizado con FlashAttention para mayor eficiencia computacional [5]. Dos técnicas habilitantes son centrales: la **cuantización**, que reduce el ancho de bits de los pesos (p. ej. INT8/INT4 o FP8/FP4) disminuyendo la huella de memoria y la latencia en hardware limitado —Husom et al., al analizar 28 LLMs cuantizados en una Raspberry Pi 4 (4 GB de RAM), reportan que las variantes q3/q4 recortan el consumo energético hasta 79% frente a FP16 (p. ej. Llama3.2-1B-instruct baja de 17.60 J/token en FP16 a 3.75 J/token en q3_K_S) [6]—, y la **destilación de conocimiento** (knowledge distillation), que transfiere las capacidades de un modelo “maestro” grande a un “estudiante” pequeño preservando gran parte de su desempeño con menor costo computacional [11]. En conjunto, un LLM compacto y cuantizado ejecutado localmente puede proveer la deliberación semántica del NPC con latencia acotada, mientras GOAP asume la ejecución táctica cuadro a cuadro, cerrando el círculo de la arquitectura híbrida de dos niveles que fundamenta esta tesis.

Referencias

Fuentes primarias obligatorias

[1] Puerta-Merino, I., & Sabater-Mir, J. (2025). *LLM Reasoner and Automated Planner: A new NPC approach*. arXiv preprint arXiv:2501.10106v1. (También publicado como DOI 10.3233/FAIA240443, IOS Press / Frontiers in Artificial Intelligence and Applications.)

[2] Kang, H., Zhang, Q., Cai, H., Xu, W., Krishna, T., Du, Y., & Weissman, T. (2025). *Win Fast or Lose Slow: Balancing Speed and Accuracy in Latency-Sensitive Decisions of LLMs*. arXiv preprint arXiv:2505.19481. (Publicado en NeurIPS 2025.)

Fuentes complementarias sobre LLMs compactos, cuantización, destilación y latencia

[3] Belcak, P., Heinrich, G., Diao, S., Fu, Y., Dong, X., Muralidharan, S., Lin, Y. C., & Molchanov, P. (2025). *Small Language Models are the Future of Agentic AI*. arXiv preprint arXiv:2506.02153. NVIDIA Research.

[4] Abdin, M., et al. (2024). *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone*. arXiv preprint arXiv:2404.14219. Microsoft.

[5] Zhang, P., Zeng, G., Wang, T., & Lu, W. (2024). *TinyLlama: An Open-Source Small Language Model*. arXiv preprint arXiv:2401.02385.

[6] Husom, E. J., Goknil, A., Astekin, M., Shar, L. K., Kåsen, A., Sen, S., Mithassel, B. A., & Soylu, A. (2025). *Sustainable LLM Inference for Edge AI: Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency*. arXiv preprint arXiv:2504.03360; ACM Transactions on Internet of Things, 6(4), Art. 28 (DOI 10.1145/3767742).

[7] Game Developer / McIntosh, T. (s.f.). *Building the AI of F.E.A.R. with Goal Oriented Action Planning*. gamedeveloper.com. [Fuente técnica de industria; sintetiza Orkin, J. (2006), "Three States and a Plan: The AI of F.E.A.R.", GDC.]

[8] Uggeli (s.f.). *Building Better AI: Hard-Won Lessons from Designing a GOAP Action Template System*. Medium. [Fuente técnica de industria.]

[9] Autores varios (2025). *CoreThink: A Symbolic Reasoning Layer to reason over Long Horizon Tasks with LLMs*. arXiv preprint arXiv:2509.00971.

[10] Autores varios (2025). *LLM-Powered AI Agent Systems and Their Applications in Industry*. arXiv preprint arXiv:2505.16120.

[11] Xu, X., et al. (2024). *A Survey on Knowledge Distillation of Large Language Models*. arXiv preprint arXiv:2402.13116.

[12] Autores varios (2025). *Using Language Models as Closed-Loop High-Level Planners for Robotics Applications: A Brief Overview and Benchmarks*. arXiv preprint arXiv:2511.07410. [Contexto sobre el patrón jerárquico LLM+planificador clásico.]

[13] Leviathan, Y., Kalman, M., & Matias, Y. (2022/2023). *Fast Inference from Transformers via Speculative Decoding*. arXiv preprint arXiv:2211.17192. (Google; ICML 2023.)

[14] Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., & Jumper, J. (2023). *Accelerating Large Language Model Decoding with Speculative Sampling*. arXiv preprint arXiv:2302.01318. (DeepMind.)

Nota sobre fuentes: Se consultaron y verificaron directamente en arXiv las dos fuentes primarias obligatorias: **arXiv:2501.10106** (Puerta-Merino & Sabater-Mir, 2025; título y autores confirmados) y **arXiv:2505.19481** (Kang et al., 2025; sus resultados, la lista completa de autores y el diseño de FPX se verificaron contra el HTML de arXiv y el PDF de las actas de NeurIPS 2025). **Discrepancias detectadas respecto al encargo:** la lista de autores de [2] incluye a **Tushar Krishna** (omitido en las instrucciones) y la fecha correcta de publicación es **mayo de 2025**, no 2024. El texto completo del paper [1] no fue accesible directamente en arXiv (solo el abstract); sus detalles metodológicos (módulos Reasoner/Planner/Interface, PDDL, "The Firefighter Problem") se tomaron de un resumen académico secundario verificable y del abstract oficial, marcando esa limitación. Las fuentes complementarias [3]–[6] y [9]–[14]

se verificaron por abstract/PDF en arXiv y sus venues; [7] y [8] son fuentes técnicas de industria sin revisión por pares, usadas solo para el funcionamiento y las limitaciones prácticas de GOAP.